



Kubernetes Networking (Expanded Overview)

1 Service

A **logical abstraction** that defines a stable endpoint (ClusterIP, NodePort, LoadBalancer, or ExternalName) to **expose Pods**.

◆ Key Features:

- Provides a virtual IP (VIP) for accessing dynamic sets of Pods.
- Automatically load-balances traffic between matching Pods (based on **selector**).



Example:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

This exposes Pods labeled **app: my-app** on port 80.

2 Endpoints

A **list of actual Pod IPs** (and ports) matched by a Service's selector.

◆ Key Features:

- Created and updated **automatically** by Kubernetes.

- Services use Endpoints to route traffic.

Example:

If your Service targets `app: nginx` and two Pods have IPs `10.0.1.2` and `10.0.1.3`, the Endpoints object would look like:

subsets:

- addresses:

- ip: 10.0.1.2

- ip: 10.0.1.3

ports:

- port: 8080

3 **Ingress**

Defines **HTTP(S) routing rules** for external traffic to reach internal **Services**.

◆ **Key Features:**

- Operates at **Layer 7 (HTTP/HTTPS)**.
- Allows **path-based or host-based routing**.

Example:

```
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
```

```
metadata:
```

```
  name: web-ingress
```

```
spec:
```

```
  rules:
```

```
    - host: example.com
```

```
      http:
```

```
        paths:
```

```
          - path: /app
```

```
            pathType: Prefix
```

```
            backend:
```

```
              service:
```

```
                name: app-service
```

```
                port:
```

```
                  number: 80
```

Routes `http://example.com/app` to the `app-service`.

4 Ingress Controller

A **specialized controller** that watches Ingress resources and **fulfills them**, typically using a reverse proxy.

♦ Popular Ingress Controllers:

- NGINX
- Traefik
- HAProxy
- Istio (Gateway)

Example:

When you install the NGINX Ingress Controller, it watches for new Ingress resources and configures the NGINX proxy accordingly to serve requests.

5 NetworkPolicy

Defines **rules to allow or deny traffic** between Pods, namespaces, or external endpoints.

♦ Key Features:

- Operates at **Layer 3/4** (IP/Port).
- Enforced by the CNI plugin (e.g., Calico).

Example:

Allow traffic **only from Pods in the same namespace**:

apiVersion: networking.k8s.io/v1

```
kind: NetworkPolicy
metadata:
  name: allow-same-namespace
spec:
  podSelector: {}
  ingress:
    - from:
      - podSelector: {}
```

6 CNI Plugins (Container Network Interface)

Provide the **underlying network implementation** for Pods, supporting IP allocation, routing, and network policy enforcement.

♦ Common CNI Plugins:

- **Calico** – Network policy + BGP routing
- **Flannel** – Simple overlay network
- **Cilium** – eBPF-based security and observability

Example:

- **Calico** enforces NetworkPolicies and provides high-performance networking.
 - **Flannel** provides basic Pod-to-Pod connectivity using VXLAN.
-

7 Multus CNI

A meta-plugin that allows **multiple CNI networks** to be attached to a Pod (multi-homed Pods).

♦ Use Cases:

- Separate management and data networks.
- Attach a Pod to an SDN (Software-Defined Network) and Kubernetes network.

Example:

You can attach both a **Flannel network (default)** and a **custom SR-IOV network** to a Pod.

Service Mesh (e.g., Istio, Linkerd)

Provides **fine-grained traffic control**, **observability**, **encryption**, and **retries/failovers** at the **application layer**.

◆ Key Features:

- Automatic mTLS encryption between services.
- Traffic routing, retries, circuit-breaking.
- Distributed tracing and metrics.

Example with Istio:

```
apiVersion: networking.istio.io/v1alpha3
```

```
kind: VirtualService
```

```
metadata:
```

```
  name: reviews-route
```

```
spec:
```

```
  hosts:
```

```
    - reviews
```

```
  http:
```

```
    - route:
```

```
      - destination:
```

```
        host: reviews
```

```
        subset: v2
```

Routes all traffic to version **v2** of the **reviews** service.



Summary Table



Component	Description	Example or Tool
-----------	-------------	-----------------

1	Service	Stable IP to access a set of Pods	ClusterIP, NodePort, LoadBalancer
2	Endpoints	Maps Service to actual Pod IPs	Auto-generated for every Service
3	Ingress	HTTP(S) routing to Services	Path- or host-based routing
4	Ingress Controller	Implements the Ingress resource	NGINX, Traefik, Istio Gateway
5	NetworkPolicy	Controls Pod-to-Pod and Pod-to-external communication	Allow or deny traffic by IP/label
6	CNI Plugins	Implements the actual networking for Pods	Calico, Flannel, Cilium
7	Multus CNI	Enables multiple network interfaces per Pod	Combine SR-IOV + Flannel
8	Service Mesh	Enhances observability, security, and control between Pods	Istio, Linkerd
